

Feature Logic Guide

Project: Smart Classroom Energy-Saving, Safety & Asset Monitoring System

Repository: <https://github.com/3351666087/desktop-tutorial>

This document explains what the system does and how each function flows through the hardware, firmware, network and dashboard layers.

1. One-Sentence Overview

The project is a local-first smart classroom IoT service: Arduino UNO performs real-time sensing and actuator control, ESP32 connects the edge node to MQTT, and the laptop dashboard provides monitoring, configuration, analytics and AI-assisted voice control.

2. System Layers

Layer	Implementation	What It Does
Physical sensing and actuation	PIR, light, gas, flame, button, LM35, LED, buzzer, relay, fan, status light	Reads the classroom environment and changes physical outputs.
Edge control	Arduino UNO	Runs the real-time state machine and safety logic.
Gateway	ESP32	Converts UNO UART telemetry into MQTT messages and forwards commands back to UNO.
Service	Laptop Node.js dashboard	Runs MQTT broker, web dashboard, settings persistence and WebSocket updates.
Intelligence	Python AI backend + Qwen	Converts voice/text into structured timed tasks and recommends configuration from data.
User interface	Desktop and mobile browser	Shows live state, charts, controls, logs, presets and microphone input.

3. Energy-Saving Lighting Logic

Goal: turn on the classroom light only when it is useful.

Logic chain:

- PIR sensor reports whether motion is detected.
- TEMT6000 reports ambient brightness through analog value.
- UNO determines whether the room is dark enough.
- If the room is occupied and dark, UNO turns on the classroom LED.
- If the room is bright, the LED stays off even if motion is detected.
- If no occupancy is detected for more than 10 seconds, the LED turns off.
- ESP32 publishes `pir`, `light`, `dark` and `lamp` to MQTT.
- Dashboard updates the lighting card and telemetry chart.

Commercial value: this directly maps to classroom energy saving by avoiding unnecessary lighting.

4. Temperature And Ventilation Logic

Goal: ventilate the room based on temperature while avoiding abrupt or unstable fan behavior.

Logic chain:

1. LM35 temperature is read through the ESP32 ADC route.
2. ESP32 includes the temperature in telemetry for the dashboard and UNO-side logic.
3. UNO compares temperature against configured thresholds.
4. If temperature is above the fan-on threshold, relay fan power is enabled.
5. PWM value scales with temperature and can also be manually overridden.
6. Tach feedback on UNO D3 estimates RPM.
7. Dashboard displays fan state, PWM and RPM.

The system uses threshold hysteresis so the fan does not rapidly switch on and off around one temperature value.

5. Safety Alarm Logic

Goal: safety always overrides comfort or energy saving.

Trigger conditions:

- gas sensor value exceeds danger threshold,
- flame sensor detects flame,
- emergency test button is pressed,
- safety demo command is active.

Safety response:

1. UNO enters `SAFETY_ALARM`.
2. Red status light is enabled.
3. Buzzer uses intermittent alarm pattern.
4. Fan relay is forced on.
5. Fan PWM is raised.
6. Normal automatic energy-saving logic is bypassed.
7. ESP32 publishes the safety state to MQTT.
8. Dashboard shows alarm mode and warning state.

This is a key design choice: safety does not wait for the web dashboard or Qwen. It is local and deterministic on the Arduino edge node.

6. Status Light Logic

The traffic-light module gives immediate visual feedback:

Status Light	Meaning
Green	normal or lighting mode
Yellow	cooling or energy-saving mode
Red	safety alarm
Red blinking	sensor error

Status Light	Meaning
All on	manual/demo state when required

This makes the system understandable even without reading the dashboard.

7. Manual Control And Presets

The dashboard supports automatic and manual control.

Manual mode can control:

- classroom lamp,
- fan on/off,
- fan PWM,
- buzzer,
- red/yellow/green status lights.

Preset strategies include:

Preset	Purpose
Comfort	balanced daily classroom operation
Energy Saver	later fan activation and lower light limits
Presentation	stable and softer indicators for demonstration
Safety First	earlier ventilation and stronger status indication

When a setting is applied, the dashboard saves it and sends MQTT commands to the ESP32. The ESP32 forwards them to the UNO and the UNO returns command ACK.

8. MQTT And Dashboard Logic

The ESP32 publishes a complete telemetry JSON message and key topic values. The dashboard subscribes to these topics through the local broker and updates live cards, charts and logs.

Important topics:

Topic	Purpose
<code>smartclassroom/edge1/telemetry</code>	complete edge state
<code>smartclassroom/edge1/status</code>	online/offline state
<code>smartclassroom/edge1/mode</code>	current mode
<code>smartclassroom/edge1/safety/alarm</code>	safety alarm flag
<code>smartclassroom/edge1/command</code>	dashboard-to-device command
<code>smartclassroom/edge1/command_ack</code>	confirmation from ESP32/UNO path

If no telemetry arrives, the dashboard shows `waiting` or stale/offline state. This helps diagnose Wi-Fi, MQTT or UART problems.

9. Mobile Voice Control Logic

This is the strongest interactive feature.

Logic chain:

1. A phone opens the local HTTPS dashboard.
2. The user presses the microphone button and speaks a command.
3. Browser sends audio to the laptop service.
4. The persistent AI backend uses faster-whisper to transcribe the audio.
5. Qwen receives the transcript, current telemetry and allowed actions.
6. Qwen returns structured JSON tasks, not free-form text.
7. The backend validates the task schema.
8. The backend publishes MQTT commands.
9. ESP32 receives MQTT commands and forwards them over UART.
10. UNO executes the command and changes the physical circuit.

Example:

```
User voice: switch to manual mode and set the fan to maximum speed
Result: manual mode is enabled and the 12 V fan changes physically
```

This demonstrates a complete AI-to-IoT actuation loop.

10. Timed Task Logic

Qwen can generate a task timeline.

Example command:

```
Switch to manual mode in 3 minutes, set fan PWM to 180, then return to automatic mode 2
minutes later.
```

Expected structured plan:

Task	Delay	Action
Switch to manual fan control	180 seconds	set_manual
Return to automatic mode	300 seconds	return_auto

After all tasks finish, the dashboard automatically clears the timeline so the interface does not remain cluttered.

11. Data Collection And Recommendation Logic

The dashboard can collect telemetry and preference events. The AI backend uses torch to recommend future automatic thresholds.

Inputs:

- temperature,
- occupancy,
- light level,

- fan state and PWM,
- manual overrides,
- configuration events.

Outputs:

- recommended fan-on temperature,
- recommended fan-off temperature,
- recommended LED brightness limit,
- confidence score.

This gives the project practical facility-management value. The system can gradually move from fixed thresholds to data-informed operation.

12. Security And Privacy Logic

Issue	Project Response
Safety dependence on network	Safety alarm runs locally on UNO.
API key exposure	Qwen key is stored in ignored local secrets, not committed.
Audio privacy	Voice files are temporary and deleted after transcription.
LLM over-control risk	Qwen cannot send raw hardware commands; backend validates allowed actions.
LAN exposure	Current prototype is LAN-only; production should add authentication and TLS trust.

13. Why This Is More Than A Sensor Demo

Many IoT demonstrations stop at displaying sensor values. This project demonstrates:

- real physical sensing,
- local edge decision-making,
- network gateway communication,
- MQTT publish/subscribe architecture,
- web operations dashboard,
- mobile microphone interaction,
- ASR and LLM command planning,
- structured task timeline,
- physical actuator response,
- testing and GitHub-based repeatability.

The most important feature is the closed loop from human intent to physical action.